

```
public class AgendaWeb : PageModel
{
    private readonly AgendaContext _context;

    public IList<Contact> Contacts { get; set; }
}
```

```
_context.Contacts.ToListAsync();

@foreach (var item in Model.Contacts)
{
}
```

Radiografía de una Agenda Web en C#

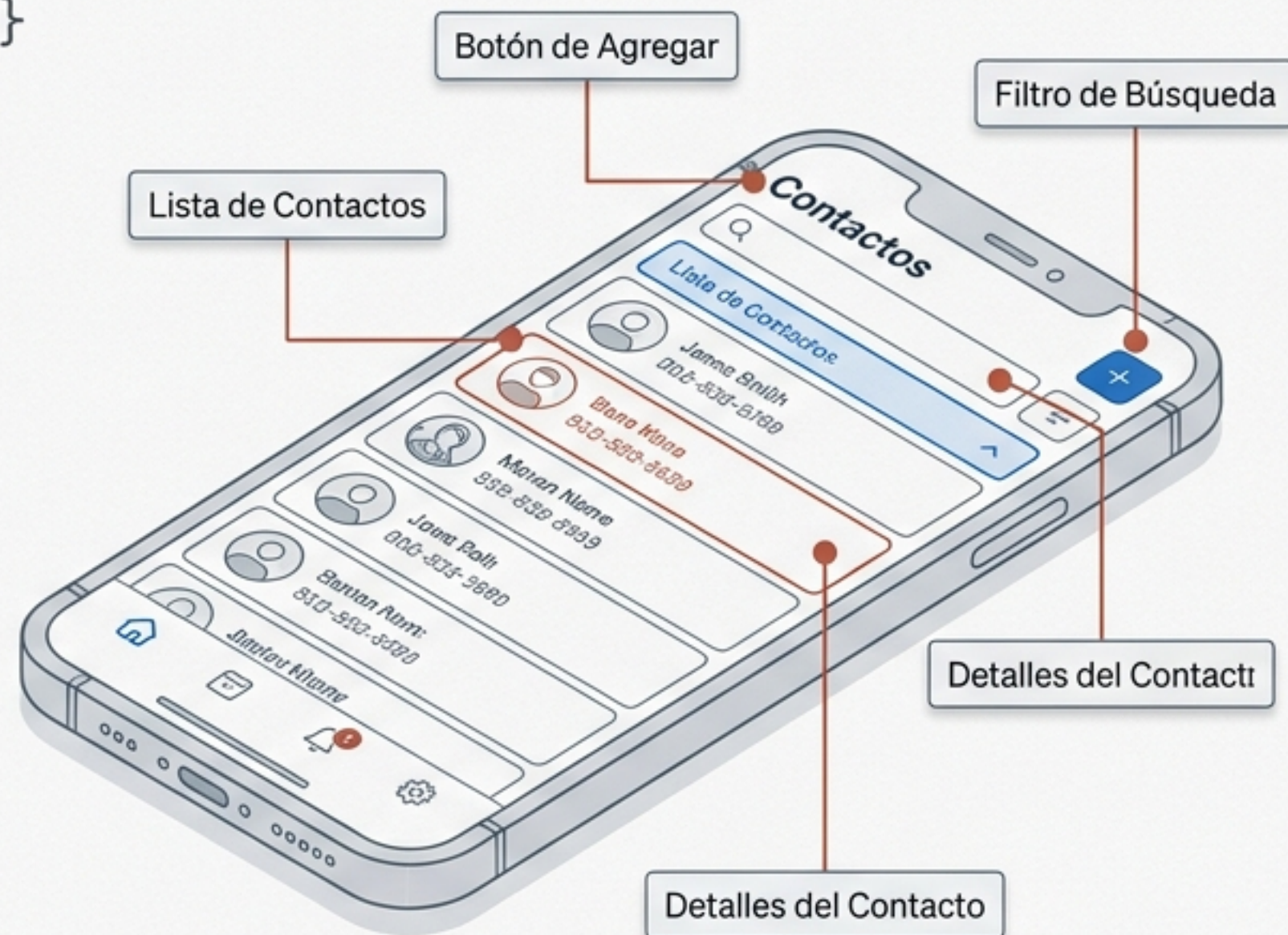
Anatomía de una aplicación ASP.NET Core Razor Pages con SQLite.

C# 11

Razor Pages

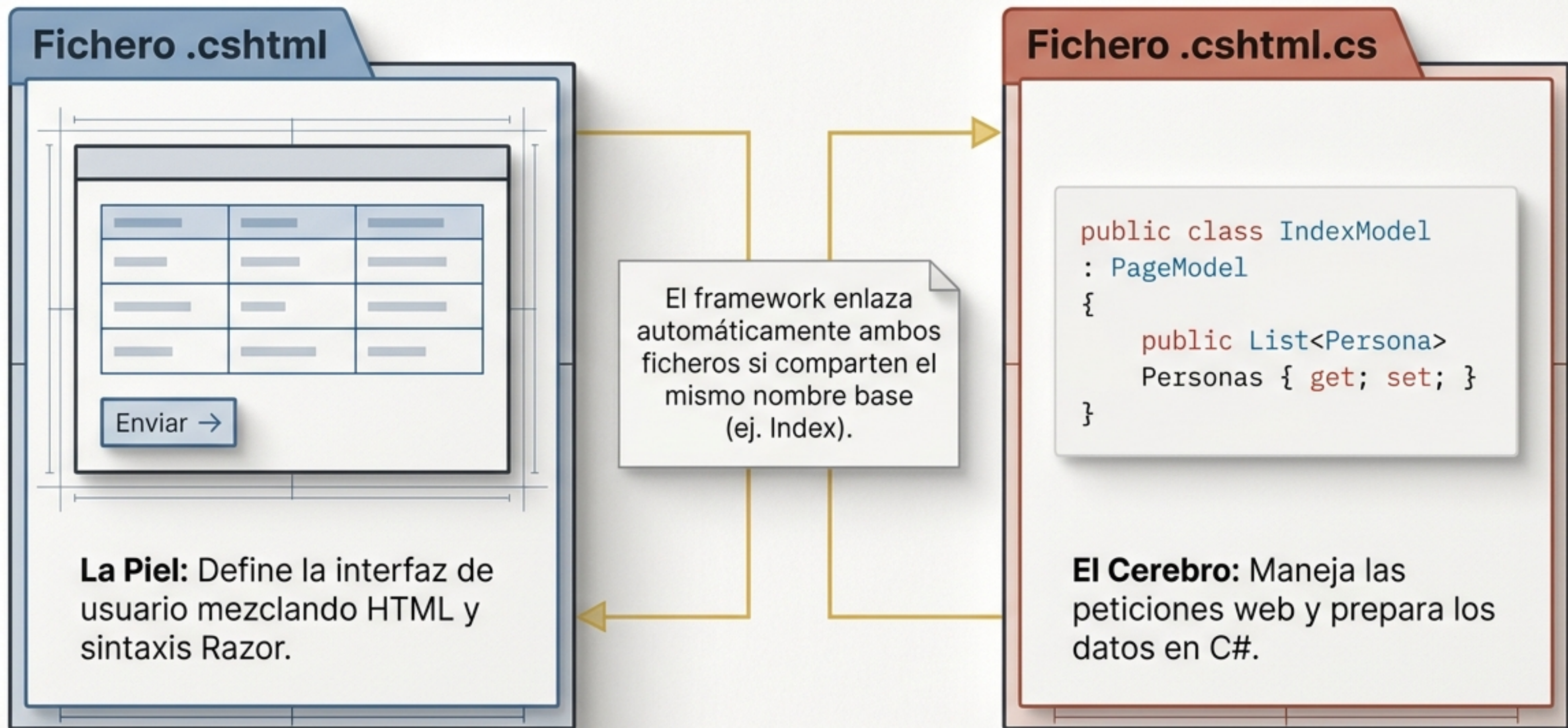
SQLite

```
<form method='post'>
    @foreach (var item in Model.Contacts) ... }
}
```

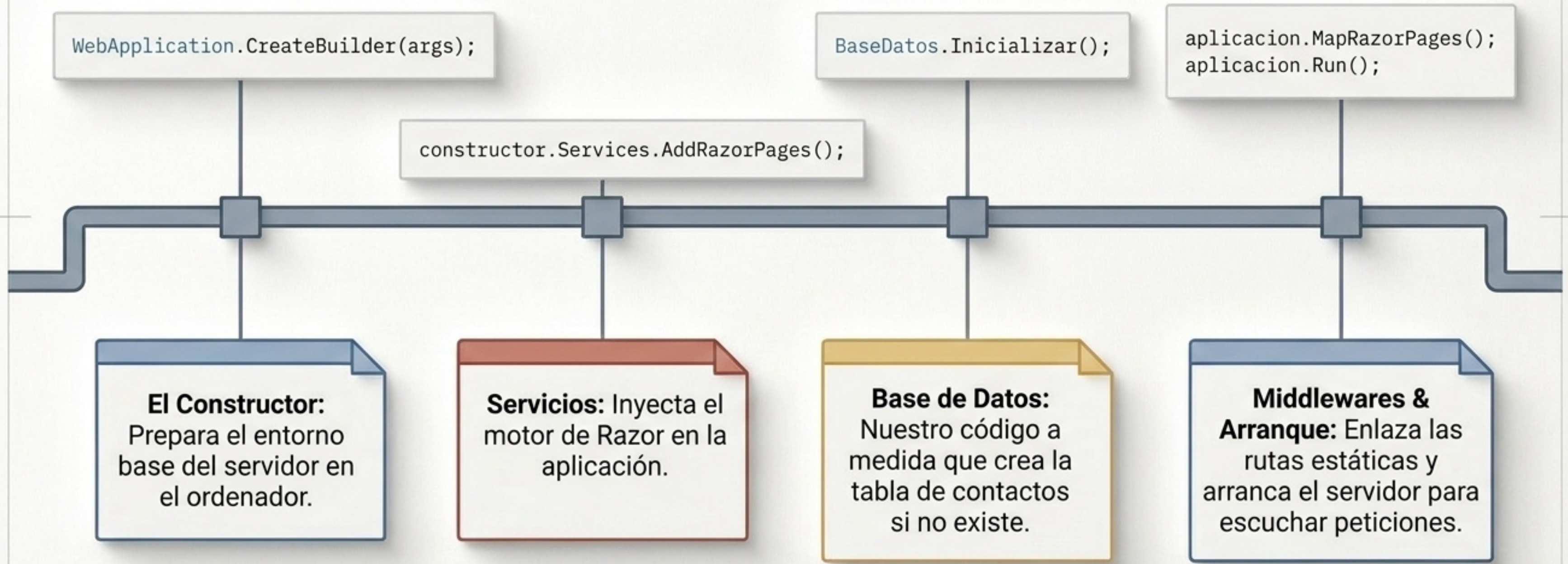


```
public IList<Contact>.ToListAsync( ;
```

...

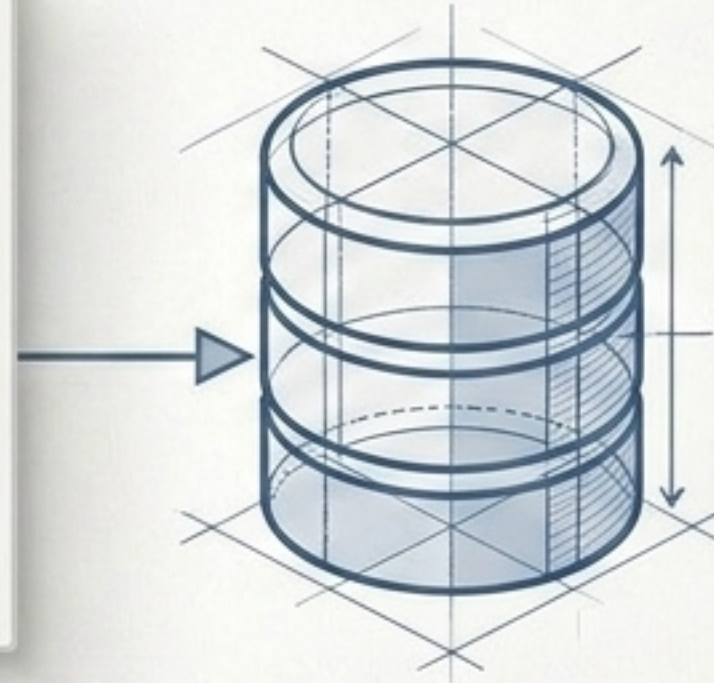


El Motor de Arranque: Program.cs



El Almacén de Datos (SQLite)

```
using Microsoft.Data.Sqlite;  
namespace AgendaContactosWeb.Datos;  
  
// Prepara la base de datos y crea conexiones  
public static class BaseDatos {  
    private static string cadenaConexion = "";  
}
```



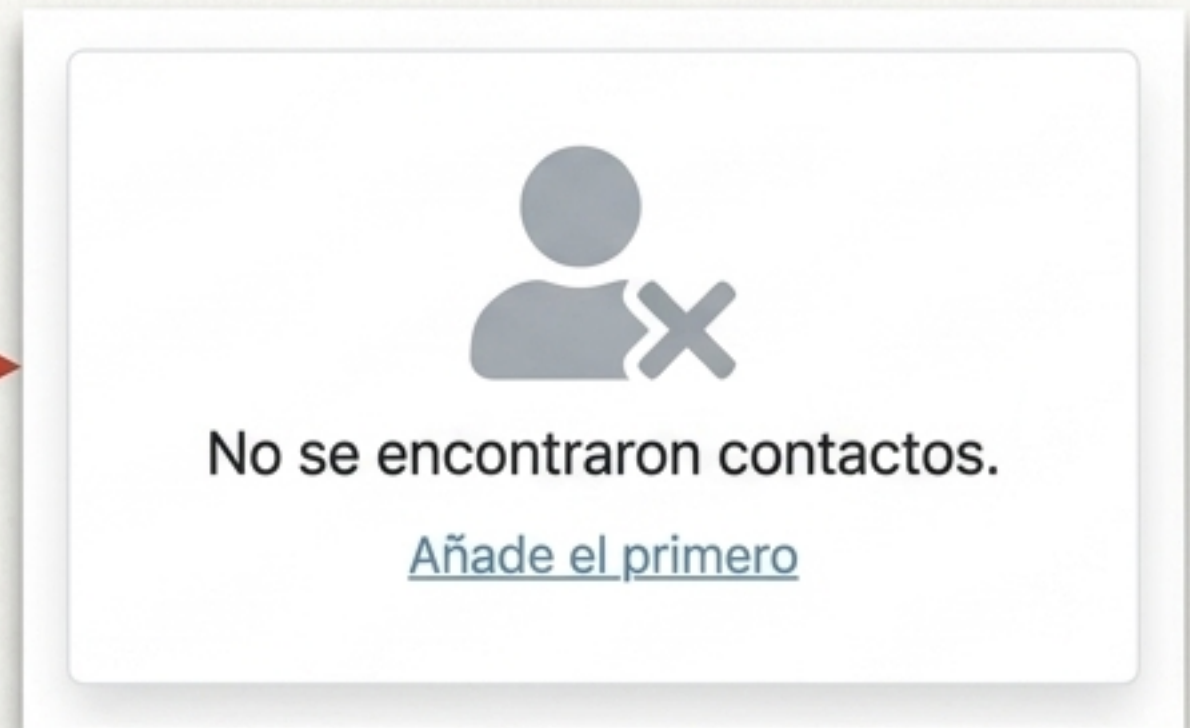
- Usamos **Microsoft.Data.Sqlite** para interactuar con la base de datos de forma ligera y directa.
- Una clase **static** gestiona la conexión: es global y no necesitamos instanciarla repetidamente.
- El método **Inicializar()** asegura que el fichero SQLite esté listo antes de que el usuario vea la primera pantalla.

La Vista Principal: Comprobando Datos (Index.cshtml)

Código Razor

```
@if (Model.Personas.Count == 0)
{
    <div class="alert text-center">
        <i class="bi bi-person-x"></i>
        <p>No se encontraron contactos.</p>
        <a asp-page="/Crear">Añade el primero</a>
    </div>
}
```

Resultado Visual



Razor evalúa el código C# en el servidor. Si la agenda está vacía, el navegador web del usuario solo recibe este bloque de HTML puro.

Inyectando Datos en la Interfaz

```
@foreach (Persona persona in Model.Personas)
```

```
<td><strong>@persona.Nombre</strong></td>
```

```
href="tel:@persona.Telefono"
```

	Name	Phone number	Action buttons
	Lorem ipsum dolor sit amet, consectetur elit.	(203) 1356-7250	 

El bucle itera sobre la lista. Por cada contacto, Razor “esculpe” una nueva fila `<tr>` inyectando los valores dinámicos en el HTML antes de enviarlo al cliente.

El Cerebro del Índice: IndexModel.cs

1

Herencia PageModel: Convierte esta clase en el controlador de la vista. Se ejecuta antes de pintar el HTML.

```
public class IndexModel : PageModel
{
    public List<Persona> Personas { get; set; } = new List<Persona>();
}
```

Herencia PageModel: Convierte esta clase en el controlador de la vista. Se ejecuta antes de pintar el HTML.

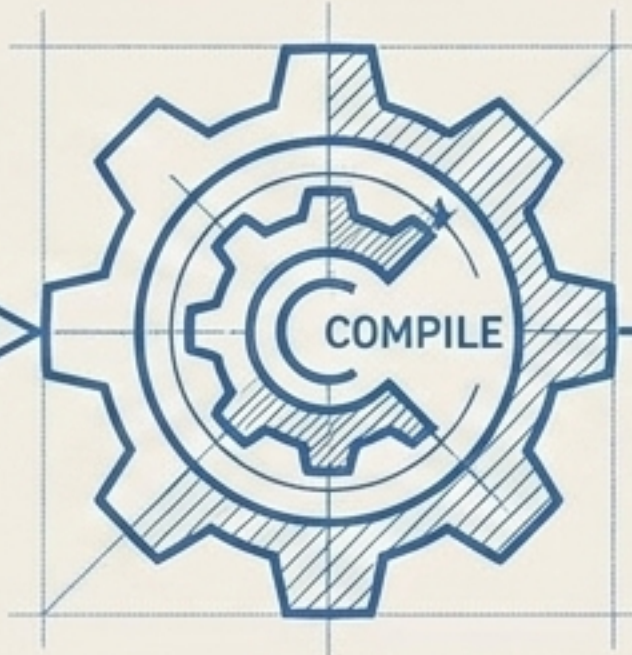
2

Propiedad Pública: Al definir la lista como pública, la hacemos accesible desde la vista (Index.cshtml) utilizando la sintaxis @Model.Personas.

Navegación y Tag Helpers

Sintaxis ASP.NET

```
<a asp-page="/Crear">  
  
<a asp-page="/Editar"  
asp-route-id="@persona.Id">
```



HTML renderizado

```
<a href="/Crear">  
  
<a href="/Editar/5">
```

Tag Helpers (atributos con prefijo **asp-**): En lugar de escribir enlaces rígidos que pueden romperse si reestructuramos los ficheros, ASP.NET calcula las URLs correctas dinámicamente basándose en la arquitectura del proyecto.

Rutas Dinámicas y Modificación

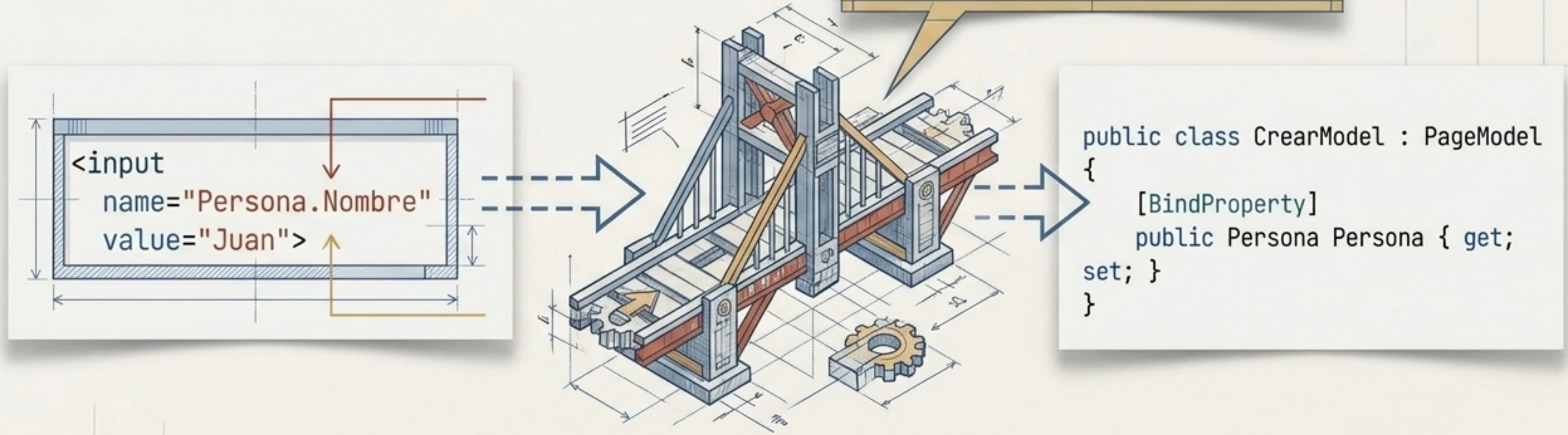
misitio.com/Editar/5

```
@page "{id:int}"  
@model EditarModel
```

Ruta Dinámica: La directiva `@page` no solo indica que es una vista, define un contrato. Exige un parámetro en la URL llamado `'id'`.

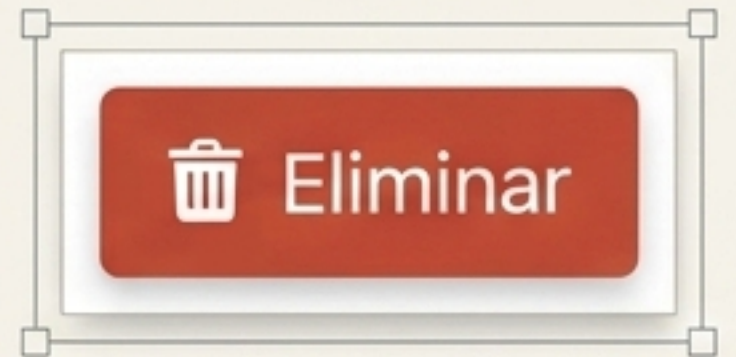
Restricción de Tipo: Añadir `':int'` asegura que solo acepte números enteros. Si un usuario intenta acceder a `/Editar/texto`, el framework devolverá un error 404 automáticamente.

La Magia del Enlace de Datos: [BindProperty]



El Enlace de Datos bidireccional. Cuando el usuario envía el formulario, el framework lee automáticamente los campos HTML entrantes y rellena las propiedades exactas del objeto Persona en C#. ¡Cero código de mapeo manual necesario!

Acciones Destructivas: Eliminar Contactos



Index.cshtml

```
<form method="post" asp-page-handler="Eliminar">
  <input type="hidden" name="id" value="@persona.Id" />
  <button class="btn btn-outline-danger" type="submit">
    <i class="bi bi-trash3"></i>
  </button>
</form>
```

Handler de Página:
Instruye a Razor para que
busque y ejecute un
método exacto llamado
OnPostEliminar() en el
Code-Behind, ignorando
las otras acciones POST.

Input Oculto: Transmite el Id del contacto de forma
silenciosa para que el servidor sepa exactamente qué
registro borrar en la base de datos SQLite.

Matriz Comparativa: Enlaces vs. Formularios

	Petición GET (Enlaces)	Petición POST (Formularios)
Acción HTML	Etiqueta <code><a></code> (ej. Abrir pantalla Editar)	Etiqueta <code><form></code> (ej. Botón Eliminar)
Intención	Solicitar y visualizar información. Operación segura sin efectos colaterales.	Modificar, crear o destruir información en el servidor.
Seguridad Web (Regla de Oro)	Los bots y motores de búsqueda siguen todos los enlaces. Si 'Eliminar' fuera un enlace GET, un rastreador web vaciaría tu agenda automáticamente.	Requiere una acción intencionada del usuario. ASP.NET protege los envíos POST automáticamente usando tokens Anti-falsificación.

UI/UX: Estilizado Profesional con Bootstrap



María Pérez

555-0192

 Editar

 Eliminar

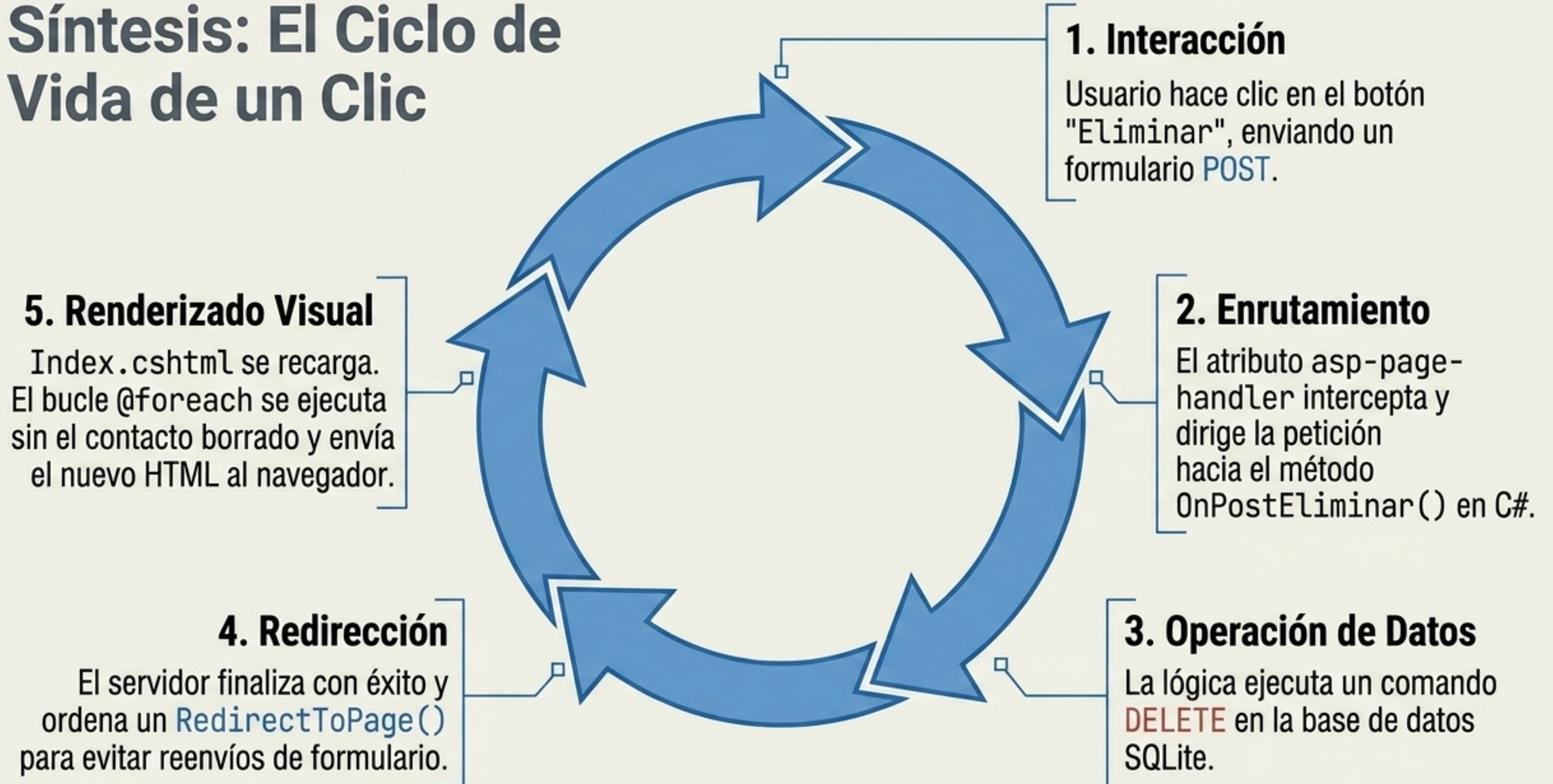
```
<table class="table table-hover align-middle shadow-sm">  
  <a class="btn btn-sm btn-outline-primary">  
    <i class="bi bi-pencil-square"></i>  
  </a>  
</table>
```

Clases Utilitarias: 'table-hover' y 'shadow-sm' aplican estilos modernos sin necesidad de escribir ficheros CSS personalizados.

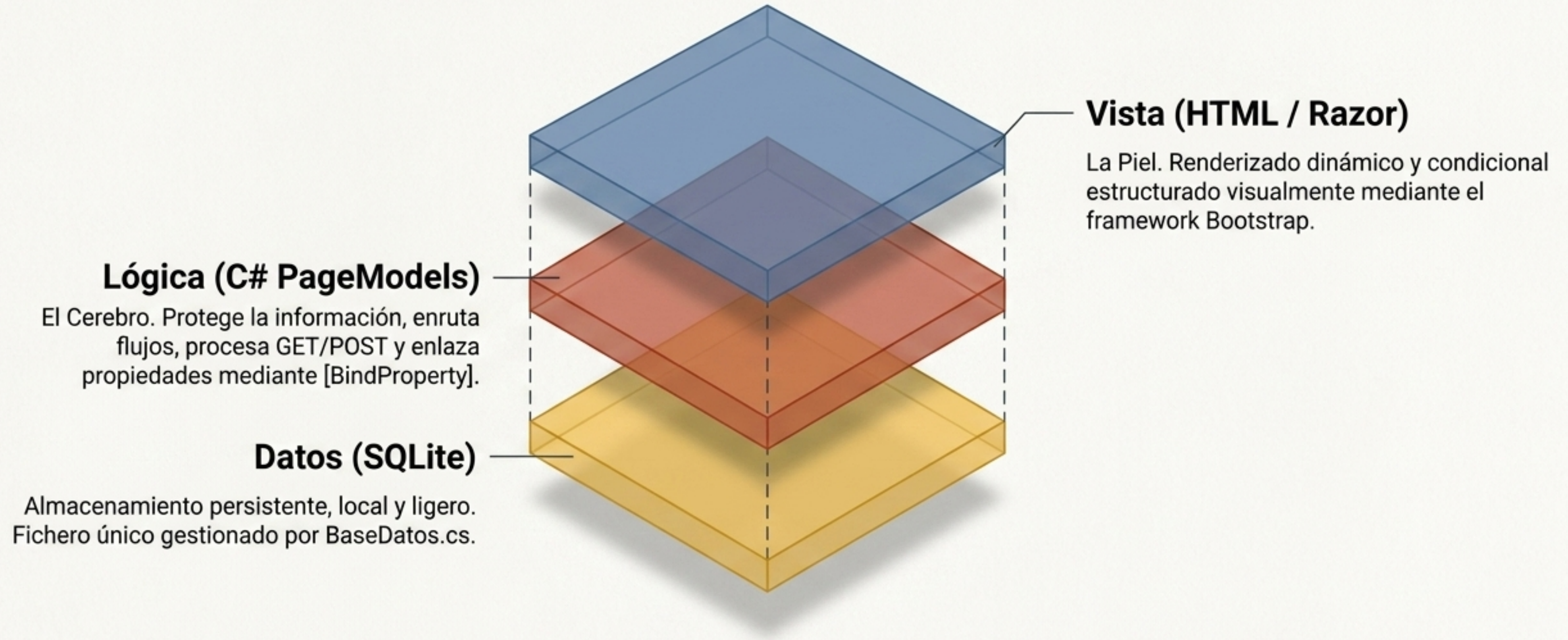
Componentes: 'btn-outline-primary' estructura y colorea el enlace pareciendo un botón interactivo.

Bootstrap Icons: Integración inmediata de iconografía vectorial mediante la clase 'bi'.

Síntesis: El Ciclo de Vida de un Clic



Conclusión: Arquitectura Razor Pages



Razor Pages logra una separación limpia de responsabilidades manteniendo una estructura de ficheros intuitiva para el desarrollador de C#.